01 Vega Health Platform ("Platform") Overview

This guide is intended to provide technology, operational, and AI governance professionals with a working understanding of the Vega Health technology stack and Platform architecture, as well as guidance on the required resources, supported integrations, and security configurations.

The Vega Health Platform uses Infrastructure as Code (IaaC) to deploy, manage, and update all system architecture and code. Vega Health runs on Kubernetes cluster and handles clinical data from hospital information systems to ingest, normalize, and serve through a set of secure APIs. At a high level, the Platform provides the following.

Exhibit 1. Vega Health Platform Layers

Feature	Description
Clinical Data Integration	Platform ingests HL7 feeds, EHR data ingestion for both retrospective and prospective data formats, and presents a patient-centric data model.
Secure API Layer	Platform relies on two separate APIs, one public-facing (authentication, user-facing features) and one internal (clinical data, service-to-service communication).
Vega Health Use Cases	Platform hosts a combination of AI/ML models and data processing pipelines and tooling including standardized monitoring, telemetry, and evaluations.

At each of these levels, Vega Health provides clients with tooling designed to enable the responsible, secure, and reliable deployment of AI in clinical and operational settings.

02 Architecture

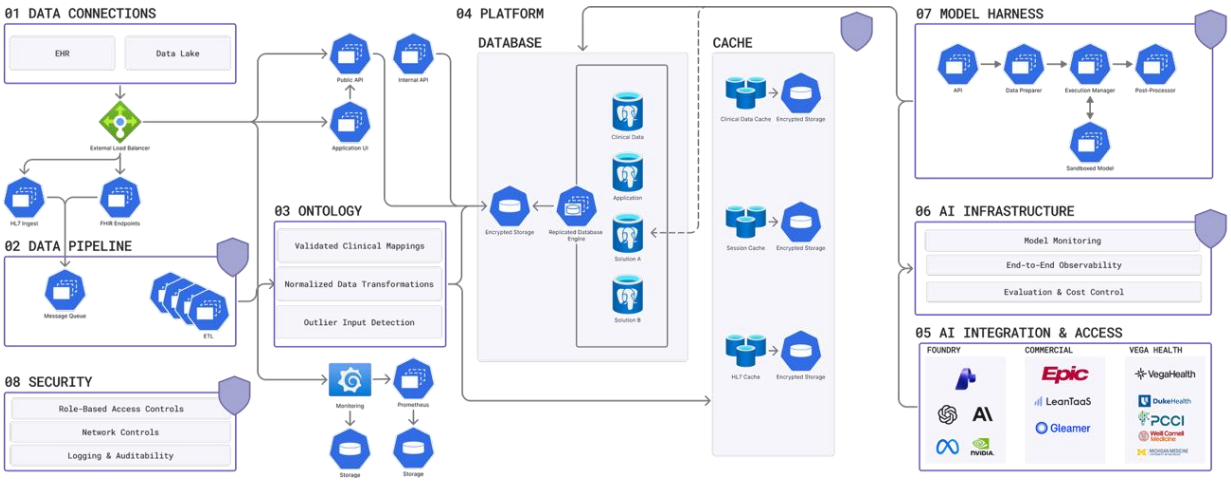
The Platform is the foundational infrastructure that supports EHR, data lake, and AI model integration. It also supports monitoring, evaluation, governance, and analytics tooling that can be applied to hosted or integrated models.

The Platform – including the clinical data integration, secure API layer, and key capabilities (e.g., hosted models, evaluation, monitoring) – consists of a service-

oriented architecture providing enterprise-class security controls using industry-leading service mesh.

This section provides an overview of the Vega Health Platform architecture, including integrations, data ingestion, ontology and data grouping, model harness and hosting, and ongoing monitoring capabilities.

Exhibit 2. Vega Health Architecture Diagram



Key components of the architecture include:

01 Data Connections. Vega Health ingests data from the client EHR [via both FHIR and HL7], as well as any data lake, dashboard, or other applications desired. Data is reconciled and assigned a single identifier.

02 Data Pipeline. Vega Health's ETL process monitors HL7 broadcasts to identify key events in patient encounters. Based on these signals, Vega Health queries FHIR APIs to augment the data necessary to run an AI model.

03 Ontology. Vega Health's proprietary ontology includes standardized data elements consisting of grouped labs, orders, vitals, and encounters. This allows clients to run AI models and monitoring on standardized system-wide data definitions and detect outlier inputs that could lead to model performance issues.

04 Platform. The Platform provides enterprise architecture necessary to build, host, run, and govern AI models.

05 AI Integration & Access. Vega Health integrates with any LLM (open- or closed-weight) including models available on Microsoft Foundry. Vega Health also provides access to a Marketplace of validated models developed and deployed in leading health systems. External models validated by Vega Health running in the client's EHR or run in

the client's network can also be integrated into Vega Health's AI infrastructure for evaluation, monitoring, and governance.

06 AI Infrastructure. Vega Health provides model monitoring, end-to-end observability (across both model features/inputs and model outputs/outcomes), and evaluation and cost control tooling to optimize value across the AI portfolio.

07 Model Harness. For AI models hosted on the Platform, Vega Health provides a model harness and solution tooling, including data access controls, logging, configuration, output processing and UI, and analytics tooling.

08 Security. Vega Health provides network security configurations, role-based access controls, and auditable logs at each level of the Platform – from data pipeline to internal API to individual solutions.

The following sections provide more details on these components, as well as the required resources, integrations, and security controls.

03 Required Resources

For clients deploying the Platform on Azure, the following resources are provisioned via an IaaC workflow.

Exhibit 3. Provisioned Resources

Resource Provider	Backing Resources
Microsoft.Network	VNet, subnets, NSGs, NAT Gateway, VPN Gateway, Private DNS zones/resolver
Microsoft.Compute	AKS node VMs, Disk Encryption Set
Microsoft.ContainerService	AKS cluster, node pools
Microsoft.KeyVault	App-facing Key Vault, CMK Key Vault
Microsoft.ManagedIdentity	AKS, cert-manager, Loki user-assigned identities
Microsoft.OperationalInsights	Log Analytics workspace
Microsoft.Insights	Diagnostic settings, metric alerts, action groups
Microsoft.Authorization	Role assignments, Azure Policy assignments
Microsoft.DBforPostgreSQL	PostgreSQL Flexible Server *(optional)*
Microsoft.Cache	Redis Cache *(optional)*
Microsoft.Storage	Loki archive storage account
Microsoft.Quota	vCPU / VM-family quota requests (via azapi)

For clients deploying the Platform locally, an alternative IaaC workflow will be utilized.

04 Infrastructure & Storage Requirements

To run the Platform in Staging and Production environments, the following resources (including compute, memory, and storage) are required.

Exhibit 4. Required Resources & Specifications

Resource	Instances	Notes & Specifications
AKS Cluster	1	Base/Istio
User Node Pools	2+ nodes	Recommend at least 8 vCPUs per node and at least 16 GB of RAM per node
Public or Private Network	1	For remote network access, Vega Health will require Azure Virtual Desktop (AVD), VPN access, or an allowlisted IP range.
Azure Key Vault Provider	1	Managed service
Postgres	1	Optional managed service
Cache	3	Optional managed service
Prometheus	1	Optional managed service
Grafana	1	Optional managed service

Exhibit 5. Staging Environment Resources and Compute/Memory/Storage Requirements

Pool	VM Size	vCPUs	RAM (GB)	Storage (GB)
Agent	Standard_D8_v5	2	8	75
General	Standard_D8_v5	8	32	0
Storage	Postgres			64
	Cache			16
	RabbitMQ			4
	Import			64

Exhibit 6. Staging Environment Resources and Compute/Memory/Storage Requirements

Pool	VM Size	vCPUs	RAM (GB)	Storage (GB)
Agent	Standard_D8_v5	2	8	75
Database	Standard_D8_v5	8	64	0
Application	Standard_D8_v5	4	16	0
Workload	Standard_D8_v5	8	64	0
Storage	Postgres			256
	Cache			128
	RabbitMQ			16
	Import			64

In addition to the above resources, Vega Health recommends baseline security features be enabled.

Exhibit 7. Recommended Security Features

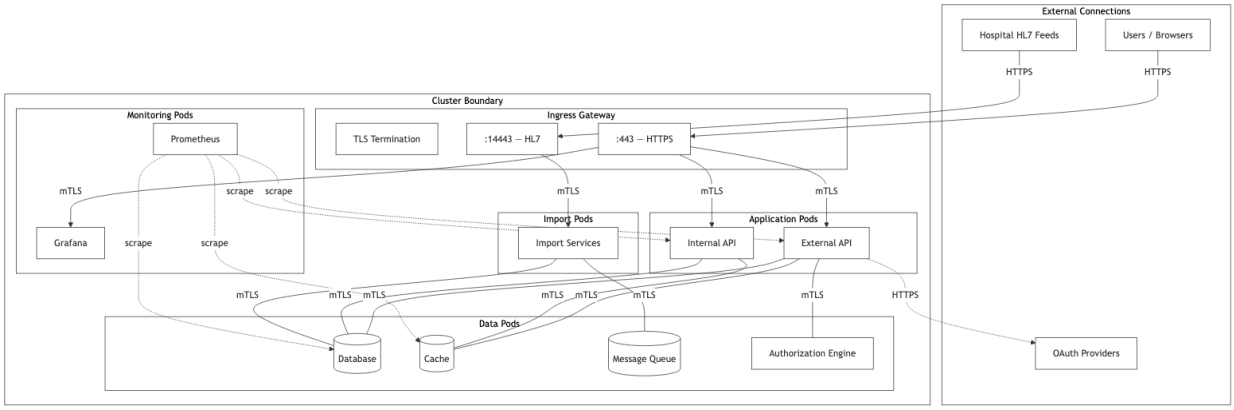
Feature	Notes & Specifications
Firewall	Application-level permissions are embedded within the service mesh. If the client wishes to place a firewall in front of the Platform, it should mirror ports used in Ingress. Vega Health makes no other or more specific recommendations for baseline security settings.
Security Monitoring & Alert System	Vega Health recommends that clients employ a security monitoring and alert system covering CPU usage, memory usage, and node failure. Vega Health also suggests that clients consider using the network or application intrusion detection software of their choice. Vega Health makes no other or more specific recommendations for baseline security settings.

05 Networking Requirements

Vega Health utilizes a service mesh to enforce encryption throughout all parts of the architecture. All inter-service communication is encrypted using mutual TLS (mTLS) via an Istio service mesh. This is enforced in **strict mode** across all application namespaces, meaning plaintext connections between services are rejected, not just discouraged.

Incoming traffic is managed via Istio routes and the Kubernetes Gateway API. Service definitions in Helm charts specify routes to specific services in the application, and these routes are automatically generated in Istio Gateway. Application pods can currently make outbound connections to external services. Formal egress policies are in place to restrict outbound traffic to a defined allowlist for annotated pods, with additional granular restrictions planned as part of the move toward a zero-trust network model. Currently, specific application pods are limited in their external outbound connections.

Exhibit 8. Vega Health Ingress & Egress Policies



Vega Health has also set traffic management to the following specifications.

Exhibit 9. Traffic Management Policies

Component	Policies
Namespaces	All Platform namespaces are annotated to enable Istio STRICT mode. Under these settings, all traffic between mesh services must be encrypted with mutual TLS and two-way authentication. Plaintext or standard TLS will be rejected.
Pods	All pods in Vega Health namespaces use mTLS to communicate with each other. This setting is required and cannot be overridden by a given service's definition or code.
Egress	Future updates will continue to tighten and restrict application outbound connections. Vega Health Use Cases will all limit egress to an explicit allowlist, whereas other pods will eventually include these modifications.

At a more granular level, the following network address space allocations are required to deploy the Platform.

Exhibit 10. Network Space Allocations

Allocation	Required For	Notes
VNet address space	Whole deployment	Must not overlap any network this VNet will ever peer with or VPN into
AKS pod CIDR (Azure CNI Overlay)	AKS cluster	Must not overlap the VNet's own address space as the overlay pod IPs are routed outside normal VNet address space
AKS service CIDR + DNS service IP	AKS cluster	DNS service IP must fall inside the service CIDR
VPN client address pool *(optional)*	P2S VPN *(optional)*	If installed, must not overlap any other range in this list
On-premises address spaces routed via S2S *(optional)*	Site-to-Site VPN *(optional)*	If installed, also scopes NSG ingress rules dropped, not misrouted
Remote hub VNet's address space (optional)	VNet peering *(optional)*	If installed, needed to confirm no overlap before peering is enabled

06 Supported LLMs

Vega Health supports integrations with HIPAA-compliant Large Language Models (LLMs) available via direct commercial LLM APIs (such as Anthropic/Claude API or OpenAI API), via Microsoft Azure AI Foundry (commercial frontier and open-weight models), or models hosted directly within a client's environment. Commercial APIs (including AI Foundry) must be contracted by the client and governed by a BAA ensuring HIPAA compliance, security of PHI, and zero data retention.

Models are selected on a task-by-task basis within the platform.

07 Required Data Integrations

Both HL7 and FHIR endpoint integrations are needed to connect a new customer's EHR system to the Platform.

The Platform receives clinical data through two channels:

1. **HL7 v2 messages** — real-time ADT (admission/discharge/transfer) and pharmacy (MAR) events pushed by the EHR to Vega Health ingest endpoint.

2. **FHIR R4 API** — on-demand and incremental pulls of structured patient data (encounters, observations, documents, orders) from EHR's FHIR server.

Both channels are required. HL7 drives real-time event awareness; FHIR provides the full clinical record for each patient.

Exhibit 11. HL7 Connection Details

Property	Value
Protocol	HTTP POST (not MLLP)
Port	14443 (configurable)
Endpoint	hl7.example.com/hl7:14443 (configurable)
Content-Type	x-application/hl7-v2+er7
HL7 version	v2.5 (refer to implementation team for alternate version)
Authentication	HTTP Basic Auth (username + password) *Strongly recommended but not required*
TLS	Handled at the network/mesh layer; coordinate with Vega Health on transport security
Max message size	5 MB

Vega Health connects to customer's FHIR R4 server (Epic, Cerner, or equivalent) to fetch structured clinical data for each patient. The customer must provision API access and supply credentials.

Exhibit 12. FHIR Authentication Requirements

Property	Value
FHIR version	R4
Base URL	Provided by customer
Transport	HTTPS
Authentication	Epic Backend Services OAuth 2.0 (JWT Bearer — RS384)
Rate limiting	Vega Health respects 429 Too Many Requests with exponential backoff

08 Security

Vega Health's engagements are centered on the delivery of enterprise architecture installed behind the firewall, managed by the client, and operated on client resources. The technology and services delivered or utilized by Vega Health are governed by the following legal documents:

The Platform, its data ingestion and API endpoints, and the mechanisms for hosting clinically validated AI/ML models that are intentionally designed to maximize security and control of an organization's network and information security teams. Security in the Platform is not a single boundary, but a series of layers that each enforce their own controls. Vega Health made the following design choices in the development of the Platform and Solutions.

Exhibit 13. Security Principles

Principle	Notes & Specifications
Least Privilege	Any program or algorithm supporting or running on the Platform is granted only strictly necessary permissions to complete a task. If those permissions are too expansive, then the scope of the program may be reduced to limit risk. Permissions are limited via API entitlement grants to reduce data access, annotations within the helm chart to prevent egress, separate namespaces, and isolation of models from the underlying Use Case and Platform.
Low/Zero-Trust	Any part of Vega Health or the health system may be compromised. Therefore, Vega Health sets the expectation that no code should be assumed to be secure by default, and a series of control layers must be in place to limit any vulnerability. The service mesh requires that all communication between services, or between services and databases, always use TLS encryption with at least AES 256-bit cipher, and must always use strong credentials. These credentials are automatically rotated between services to reduce any exposure timeframe. Payloads are validated between components before processing. Any plaintext connections between pods are rejected by default.
Limited Blast Radius	If a component of Vega Health's code could be compromised or cause damage in the case of a security incident, Vega Health has taken steps to ensure that the scope of any damage is limited. All data must be request via an API either internally or externally. Low-trust and least

Principle	Notes & Specifications
	privileges limit the data exposure limits lateral movement of a malicious actor. Additionally, Vega Health is using modern technology and practices, keeping code in small testable components, and avoiding complex monoliths that could individually compromise system integrity. Additionally, sensitive information, such as PII, is transmitted and stored only where strictly necessary.
No Plaintext Secrets	No secrets are stored in files or displayed in logs. Secrets at rest are encrypted using at least AES 256-bit cipher, except for secrets mounted into containers (however, the container underlying the storage system must be encrypted). These secrets are maintained via a secured Key Vault service and an external secrets operator to limit secrets at the individual pod level. These secrets are stored either as files (for certificates) or mounted as environment variables during runtime. The blast radius for secrets within the code is limited to code running within the containers and secrets are limited to what is strictly needed (Note: future upgrades will utilize distro-less containers, further reducing any unnecessary tooling).
Security-First Code Review Process	Across the CI/CD pipelines for all projects, all code changes are scanned with a SAST tool and security-focus code agents. Following the initial review, the Vega Health development and security team remediates identified vulnerabilities and completes an additional review for potential and actual security issues in all code changes.

The Vega Health API also enforces configurable rate limiting for most services. Including requests to the API, listing records, searching for results, and accessing resources. Additionally, rate limits are enforced for all login endpoints (e.g. OAuth attempts), end-user key rotations, and expensive operations such as calls to large language models. Repeated high-frequency events (such as repeated unauthorized access attempts) are logged, aggregated, and reported to the external logging system.

Additionally, Vega Health and Vega Health's internal instance of the Platform are HIPAA and SOC 2 Type II certified.

NOTE: These certifications apply to Vega Health's internal development, data management, security, and compliance processes. Once the Platform is installed at a client site, Vega Health no longer solely manages configurations necessary for certification of controls.